GPU-Accelerated Parallel BFS: Linear-Algebraic vs. Graph-First on a Single A100

Aakaash Nattanmai | Harshaan Chugh

CS 5220: Advanced Applications of Parallel Computers • Cornell University • NERSC Perlmutter (NVIDIA A100)

1. Motivation

Breadth-First Search (BFS) is the workhorse of graph analytics – shortest paths, connected components, centrality, web crawling, social-network and brain-connectome pipelines all depend on a fast BFS primitive.

Yet BFS is notoriously *hard to parallelize* on GPUs:

- Irregular, data-dependent memory access patterns.
- Severe load imbalance on skewed-degree graphs.
- Frontier sizes that vary by orders of magnitude per level.

The community has converged on **two** competing GPU paradigms. This poster compares both on identical hardware, inputs, and CSR storage so the algorithmic differences are isolated from implementation noise.

2. Two Paradigms

(a) Linear-Algebraic (SpMV-based). Express each BFS level as a sparse matrix–vector multiply on the adjacency matrix A:

$$f_{k+1} = (A \cdot f_k) \odot \neg v_k,$$

where f_k is the level-k frontier (dense or bitmap) and v_k the visited mask. Decades of sparse-LA optimization apply directly.

(b) Graph-First (Direction-Optimizing). Operate on graph topology with explicit *push* (top-down) and *pull* (bottom-up) kernels, switching adaptively based on frontier size [1]. Closer to traditional CPU BFS, avoids reformulating BFS as linear algebra.

3. Research Questions

1. Which paradigm wins on which graph topology (chain, sparse-random, dense-random, power-law)?
2. Are the classical optimizations (*warp-cooperation, bitmap frontier, push-pull*) equally beneficial when expressed as SpMV vs. as graph traversal?
3. What is the runtime cost (or savings) of the *linear-algebra abstraction* compared to a hand-tuned graph kernel?

4. Contributions

- Five SpMV-BFS kernel variants, including a fused warp-cooperative + bitmap kernel.
- Direction-optimizing graph-first BFS using Beamer's α/β heuristic.
- Apples-to-apples benchmark on a shared CSR loader, identical inputs, single A100.
- Open-source implementation; reproducible `run.sh` workflow.

5. Common Experimental Setup

Storage. Compressed Sparse Row (CSR): `row_ptr[V+1]`, `col_idx[2E]`. Edges are loaded once on the host, undirected (each edge inserted in both directions), then transferred to GPU global memory.

Hardware. Single NVIDIA A100 (40GB) on NERSC *Perlmutter*. Compute capability `sm_80`; compiler `nvcc -O2`. Restricted to `CUDA_VISIBLE_DEVICES=0` for fair single-GPU comparison.

Validation. Every variant is checked level-by-level against a reference Python BFS (`validate_bfs.py`). Only kernel time is measured (CUDA events); graph load and I/O are excluded.

Graph	$\lvert V\rvert$	$\lvert E\rvert$	Avg. deg.
medium_chain	1,000	999	2
medium_sparse	1,000	3K	6
medium_dense	1,000	50K	100
large_sparse	100,000	500K	10
large_dense	100,000	5M	100
large_powerlaw	100,000	1M	40

Synthetic test graphs generated by `generate_graphs.py` (RMAT for power-law).

6. Linear-Algebraic SpMV BFS (implemented)

Each BFS level is a masked SpMV. Five kernel variants:

- **Baseline** – one thread per row; scans neighbors, writes `new_frontier[i]=1` if any neighbor is in f_k. Simple but per-row load-imbalanced.
- **Opt 1 – Warp-Cooperative Rows.** 32 lanes share one row; `__ballot_sync()` reduces hit-bits in one instruction. Eliminates hub-vertex stragglers.
- **Opt 2 – Bitmap Frontier.** Pack f_k into `uint32_t` (1 bit/vertex); writes via `atomicOr`. 400 KB → 12.5 KB at V=100K – fits in L2.
- **Opt 3 – Push-Pull (Direction-Optimizing SpMV).** Switch between row-pull and column-push per level based on $\lvert f_k\rvert$. SpMV-side analogue of Beamer et al. [1].
- **Opt 4 – Fused Warp+Bitmap.** Warp-cooperative scan *of* a bitmap frontier – best of both worlds. Fused inner loop:

```
// 32 lanes scan one row of A; bitmap frontier
for (int e = row_start + lane; e < row_end; e += 32) {
    int j = col_idx[e];
    if (frontier_bm[j>>5] & (1u << (j & 31))) {
        atomicOr(&new_bm[i>>5], 1u << (i & 31));
        depth[i] = level; break;
    }
}
```

7. Graph-First Direction-Opt. BFS (implemented)

Beamer's direction-optimizing BFS [1]: a per-level decision between two kernels.

Top-down (push). Each thread owns one frontier vertex and pushes itself onto every unvisited neighbor. `atomicCAS` on `visited[]` prevents duplicate enqueues. Optimal when the frontier is small (early/late BFS levels).

Bottom-up (pull). Each thread owns one unvisited vertex and stops as soon as it finds *any* neighbor in the frontier. Optimal in the middle of BFS when most edges already point to the frontier.

Switching heuristic. Let m_f be the sum of out-degrees of frontier vertices; switch top-down → bottom-up when

$$m_f > \frac{\lvert E\rvert}{\alpha}, \qquad (\alpha = 14)$$

and back to top-down when $\lvert f_k\rvert < \lvert V\rvert/\beta$ with $\beta = 24$. **Implementation.** The GPU driver keeps each frontier as both a compact queue and a bitmap. Both push and pull kernels produce the next queue, next bitmap, next frontier size, and next frontier edge count entirely on device; the host copies only those two counters for the next direction decision.

- [DONE] CUDA-event timing matches the SpMV path.
- [DONE] `-dump` emits raw depths for validation.
- [DONE] `-alpha/-beta` expose switch tuning.

8. Results

Median of N=10 runs on one Perlmutter A100. The table compares the fastest linear-algebraic SpMV variant for each graph against the graph-first direction-optimizing traversal.

Graph	$\lvert V\rvert$	Avg. deg	Dep	Best SpMV variant	SpMV ms	Graph ms	GF/SpMV	TD/BU
medium_chain	1K	2	999	bitmap	18.733	32.918	1.76×	1000/0
medium_dense	1K	100	2	warpbitmap	0.557	0.697	1.25×	1/2
medium_sparse	1K	6	6	warp	0.665	0.758	1.14×	4/3
large_dense	100K	100	4	bitmap	4.438	5.085	1.15×	3/2
large_powerlaw	100K	40	3	warpbitmap	2.325	6.264	2.69×	2/2
large_sparse	100K	10	7	bitmap	1.446	1.927	1.33×	5/3

Take-aways: the best SpMV variant wins on all six inputs in this implementation. Graph-first stays within 1.14–1.33× on sparse/dense random graphs, but the power-law graph exposes higher graph-first overhead from queue construction and atomic frontier updates. The chain graph remains pathological for both paradigms because it forces 999 sequential BFS levels.

9. Multi-GPU Scaling

Today: a single A100. Natural next steps:

- 1D vertex partitioning of CSR across Perlmutter's 4-GPU nodes and beyond.
- Boundary-frontier exchange via NCCL all-to-all (or MPI).
- Strong / weak scaling on Graph500-class RMAT (V=2^{26}, E=2^{30}).
- Comm. / compute overlap with CUDA streams.

10. Real-World Graphs

Replace synthetic graphs with public benchmarks:

- SNAP: `soc-LiveJournal1`, `web-Google`, `roadNet-CA`, `wiki-Talk`.
- GAP suite: `kron, urand, web, road, twitter`.
- DIMACS road networks (push-pull stress test).

Compare against **NVIDIA cuGraph** as the production baseline.

11. Algorithmic Extensions & Open Questions

The SpMV abstraction generalizes via semirings (GraphBLAS):

- **SSSP** – min-plus semiring over weighted graphs.
- **Connected components** – label-propagation SpMV.
- **Triangle counting / k-core** – shared kernels.

Graph-first extends to delta-stepping SSSP and asynchronous label-propagation. Open questions:

- Can a fused warp+bitmap+push-pull SpMV close the gap to hand-tuned graph-first kernels?
- Do Beamer's α, β heuristics still hold on HBM3 GPUs (H100, MI300)?
- How does this compare to CPU SuiteSparse:GraphBLAS?

12. Energy Profiling

Runtime is only one axis. A natural follow-up is energy/edge using NVML:

- Bracket each BFS with `nvmlDeviceGetTotalEnergyConsumption` for ΔJ per traversal.
- Joules-per-edge ($GTEPS/W$) as a primary metric.
- Is the fastest kernel also the most energy-efficient?

13. References & Code

References

[1] S. Beamer, K. Asanović, D. Patterson. *Direction-Optimizing BFS.* SC '12.
[2] D. Merrill, M. Garland, A. Grimshaw. *Scalable GPU Graph Traversal.* PPoPP '12.
[3] J. Kepner et al. *Mathematical Foundations of the GraphBLAS.* HPEC '16.
[4] S. Beamer et al. *The GAP Benchmark Suite.* arXiv:1508.03619, 2015.
[5] NVIDIA RAPIDS. *cuGraph: GPU graph analytics.* `github.com/rapidsai/cugraph`.

Computations on NERSC Perlmutter *(NVIDIA A100, `sm_80`). Source, build scripts, validation harness:* `github.com/Harshaan-Chugh/parallel-bfs`.

LaTeX TikZposter